

Come scegliere l'algoritmo?

Giovedì 5 Marzo 2026

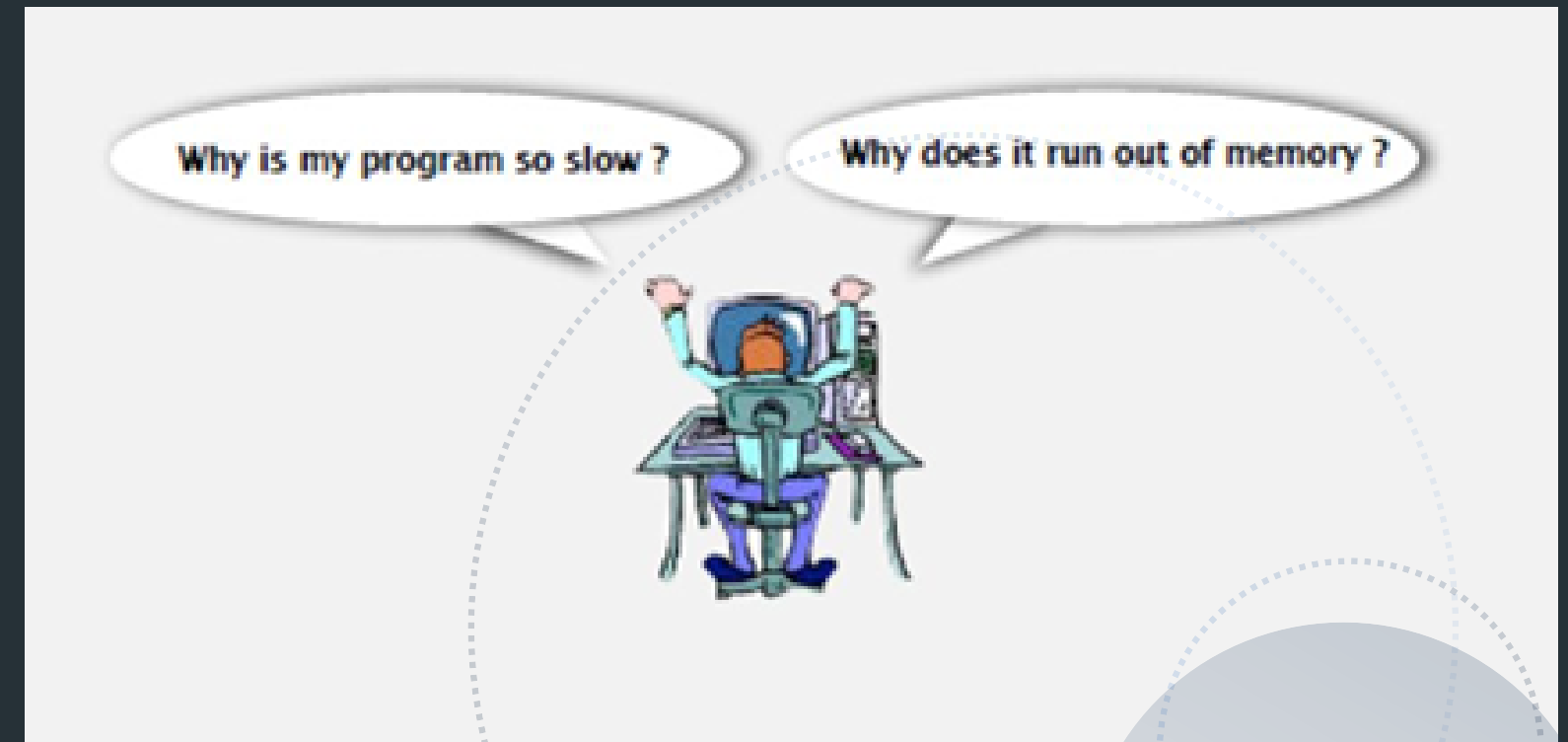
Valutazione di un algoritmo

Solitamente, si analizzano le risorse impiegate per risolvere il problema, in funzione della dimensione e della tipologia dell'input.

Le risorse sono di due tipi:

Tempo: tempo impiegato per completare l'algoritmo

Spazio: quantità di memoria utilizzata



Analisi della complessità di tempo di un algoritmo nel caso peggiore

- Per avere una valutazione oggettiva sul costo di esecuzione di un algoritmo, la valutazione non deve dipendere dal particolare dato di ingresso.
- Il costo del programma verrà valutato in funzione delle dimensioni dell'input con riferimento al caso peggiore, cioè quello in cui l'esecuzione impiega più tempo.
- Dovremo quindi di volta in volta individuare qual è il caso peggiore per quell'algoritmo, e valutare come si comporta l'algoritmo su questo input.

Il **caso ottimo** (o migliore) è invece il caso in cui l'algoritmo impiega meno

Analisi della complessità computazionale

Tempo

Dato un algoritmo si dice che esso opera in un tempo $t(n)$ se dato un input x di taglia n , l'algoritmo utilizza $t(n)$ passi per produrre il risultato.

Spazio

Dato un algoritmo si dice che esso opera in uno spazio $s(n)$ se dato un input x di taglia n , l'algoritmo utilizza $s(n)$ locazioni di memoria temporanee per produrre il risultato. Non si tiene conto dello spazio necessario per l'input e per l'output, ma solo dello spazio necessario all'esecuzione dell'algoritmo.

Ordine di complessità - Notazione O

Un algoritmo ha complessità di tempo $O(f(n))$

(e diremo che $t(n)$ ha complessità O grande di $f(n)$)

se esistono due costanti positive c ed n_0 , indipendenti da n , tali che il numero di istruzioni $t(n)$ che devono essere eseguite con un input di dimensione n verifica la seguente disuguaglianza

$$t(n) < c \cdot f(n) \quad \text{per ogni } n > n_0$$

Analogamente per $s(n)$.

La notazione O viene usata soprattutto nell'analisi del costo computazionale nel caso peggiore.

Ordine di complessità - Notazione Ω

Un algoritmo ha complessità di tempo $\Omega(f(n))$

(e diremo che $t(n)$ ha complessità Ω di $f(n)$)

se esistono due costanti positive c ed n_0 , indipendenti da n , tali che il numero di istruzioni $t(n)$ che devono essere eseguite con un input di dimensione n verifica la seguente disuguaglianza

$$c \cdot f(n) < t(n) \quad \text{per ogni } n > n_0$$

Analogamente per $s(n)$.

La notazione Ω viene usata soprattutto nell'analisi del costo computazionale nel caso migliore.

Ordine di complessità - Notazione Θ

Un algoritmo ha complessità di tempo $\Theta(f(n))$

(e diremo che $t(n)$ ha complessità Θ di $f(n)$)

se esistono tre costanti positive **c,d** ed **n_0** , indipendenti da n , tali che il numero di istruzioni $t(n)$ che devono essere eseguite con un input di dimensione n verifica la seguente disuguaglianza

$$\mathbf{c \cdot f(n) < t(n) < d \cdot f(n) \text{ per ogni } n > n_0}$$

Analogamente per $s(n)$.

La notazione Θ si usa soprattutto per indicare che il costo computazionale ha esattamente un certo ordine di grandezza per qualunque input.

soluzione per trovare il minimo

$\text{min}(S, n)$

$\text{min} = S[1]$

for $i = 2$ **to** n **do**

if $S[i] < \text{min}$ **then**

$\text{min} = S[i]$

return min

Minimo parziale

Nuovo minimo
parziale

Complessità di tempo: $n-1$ confronti

E' un algoritmo ottimale.

Complessità di tempo per l'algoritmo per il minimo

La complessità di tempo dell'algoritmo visto precedentemente è $t(n)=\theta(n)$.

La complessità di spazio $s(n)$ è costante, ovvero $\theta(1)$.

Ricerca in una lista

```
lookup( $S, n, v$ )
```

```
for  $i = 1$  to  $n$  do
```

```
    | if  $S[i] = v$  then
```

```
        | return  $i$ 
```

```
return 0
```

Algoritmo ingenuo: n confronti.

Qual è la complessità di tempo di questo algoritmo? $O(n)$

Caso di Studio: La Sottosequenza Massimale



Il Problema: Dato un array di N numeri reali, trovare la somma massima in una sottosequenza contigua.

I Casi Limite

- **E se sono tutti positivi?** La soluzione è l'array intero.
- **E se sono tutti negativi?** La soluzione è l'array vuoto (somma = 0).

Qual è la strategia algoritmica?

L'algoritmo, proposto nel 1984 da Jay Kadane, funziona perché, ad ogni passo, valuta se è meglio:

- a. Iniziare una nuova sottosequenza con l'elemento corrente (se l'elemento corrente è maggiore della somma precedente più l'elemento stesso), oppure
- b. Continuare la sottosequenza corrente.

In questo modo, riesce a trovare la somma massimale in tempo lineare $O(n)$.

- Utilizza la strategia di programmazione chiamata programmazione dinamica.

La programmazione dinamica è una tecnica utilizzata per risolvere problemi complessi suddividendoli in sottoproblemi più semplici e memorizzando i risultati di questi sottoproblemi per evitare calcoli ridondanti. Qui si applica questa strategia per calcolare la somma massimale della sottosequenza contigua in un array, utilizzando i risultati precedenti per costruire la soluzione ottimale passo dopo passo.

In particolare...

L'algoritmo mantiene una soluzione parziale (`max_ending_here`) che rappresenta la somma massimale della sottosequenza che termina nell'elemento corrente. Questo valore dipende solo dal risultato calcolato per l'elemento precedente.

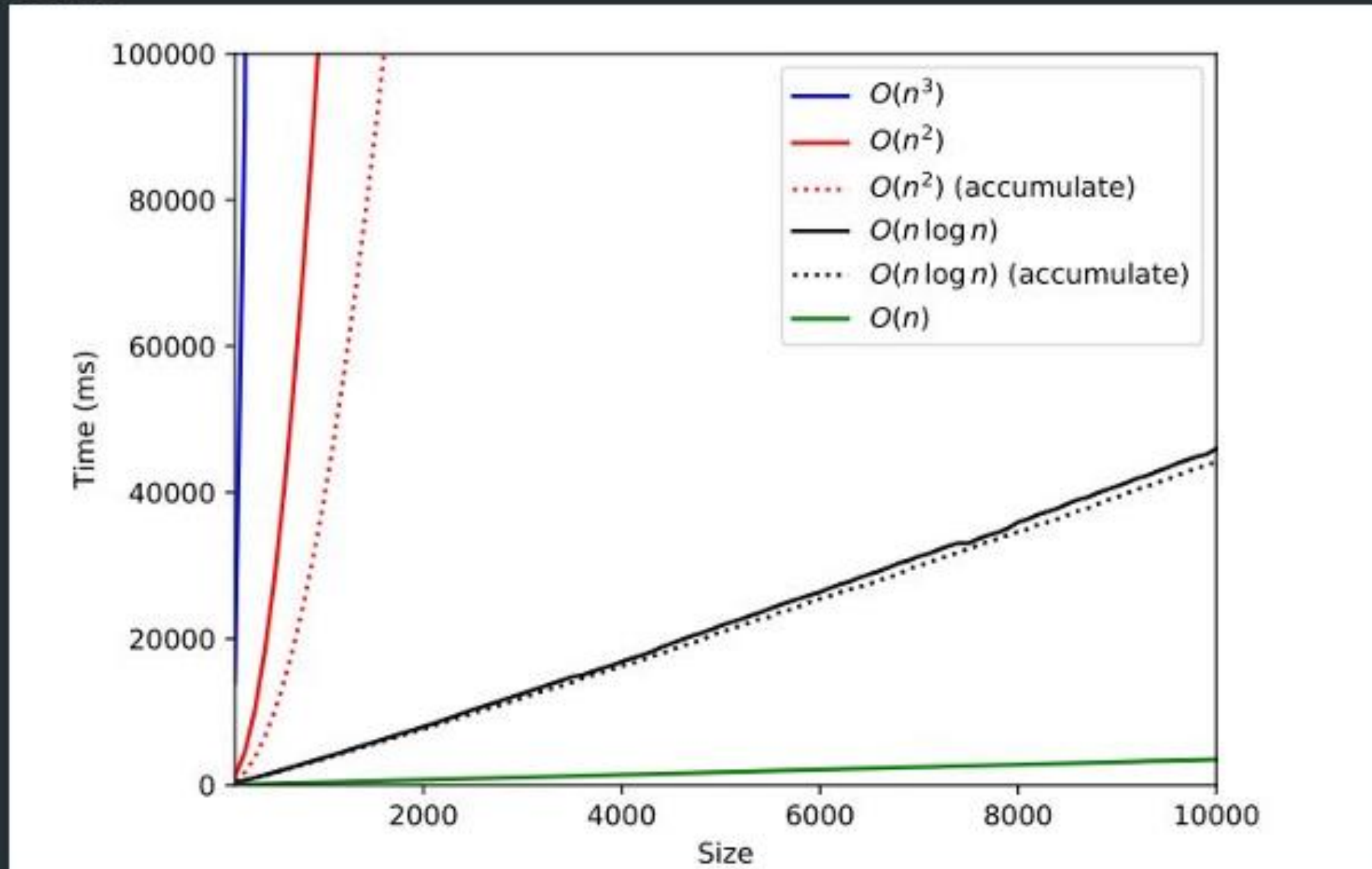
Memorizzazione temporanea: L'algoritmo memorizza solo i risultati necessari (in questo caso, `max_ending_here` e `max_so_far`), evitando di dover ricomputare i valori. In altre parole, utilizza lo **spazio ottimale** (costante $O(1)$) per tenere traccia dei valori parziali.

Vantaggi della Programmazione Dinamica per questo problema

- **Efficienza Temporale:** Grazie alla programmazione dinamica, l'algoritmo riesce a risolvere il problema in tempo lineare $O(n)$.
- **Efficienza Spaziale:** Memorizza solo una quantità costante di informazioni, risultando in una complessità spaziale di $O(1)$.

ESERCIZIO da proporre: Implementare la funzione che preso in input un vettore di N numeri reali generati casualmente calcoli la somma della sottosequenza massimale. Misurare il tempo di calcolo in secondi per valori di N pari a 100, 1000, 2000, 5000, 10000. Confrontare con le funzioni precedenti

Tempi di esecuzione



COME MISURARE IL TEMPO DI ESECUZIONE?

java

```
long start = System.currentTimeMillis();  
// Esegui codice  
long end = System.currentTimeMillis();  
System.out.println("Tempo impiegato: " + (end - start) + " ms");
```

ORDINI DI GRANDEZZA

order of growth	name	typical code framework	description	example
1	constant	<code>a = b + c;</code>	statement	add two numbers
$\log N$	logarithmic	<code>while (N > 1) { N = N / 2; ... }</code>	divide in half	binary search
N	linear	<code>for (int i = 0; i < N; i++) { ... }</code>	loop	find the maximum
$N \log N$	linearithmic	[see mergesort lecture]	divide and conquer	mergesort
N^2	quadratic	<code>for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) { ... }</code>	double loop	check all pairs
N^3	cubic	<code>for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) for (int k = 0; k < N; k++) { ... }</code>	triple loop	check all triples
2^N	exponential		exhaustive search	check all subsets

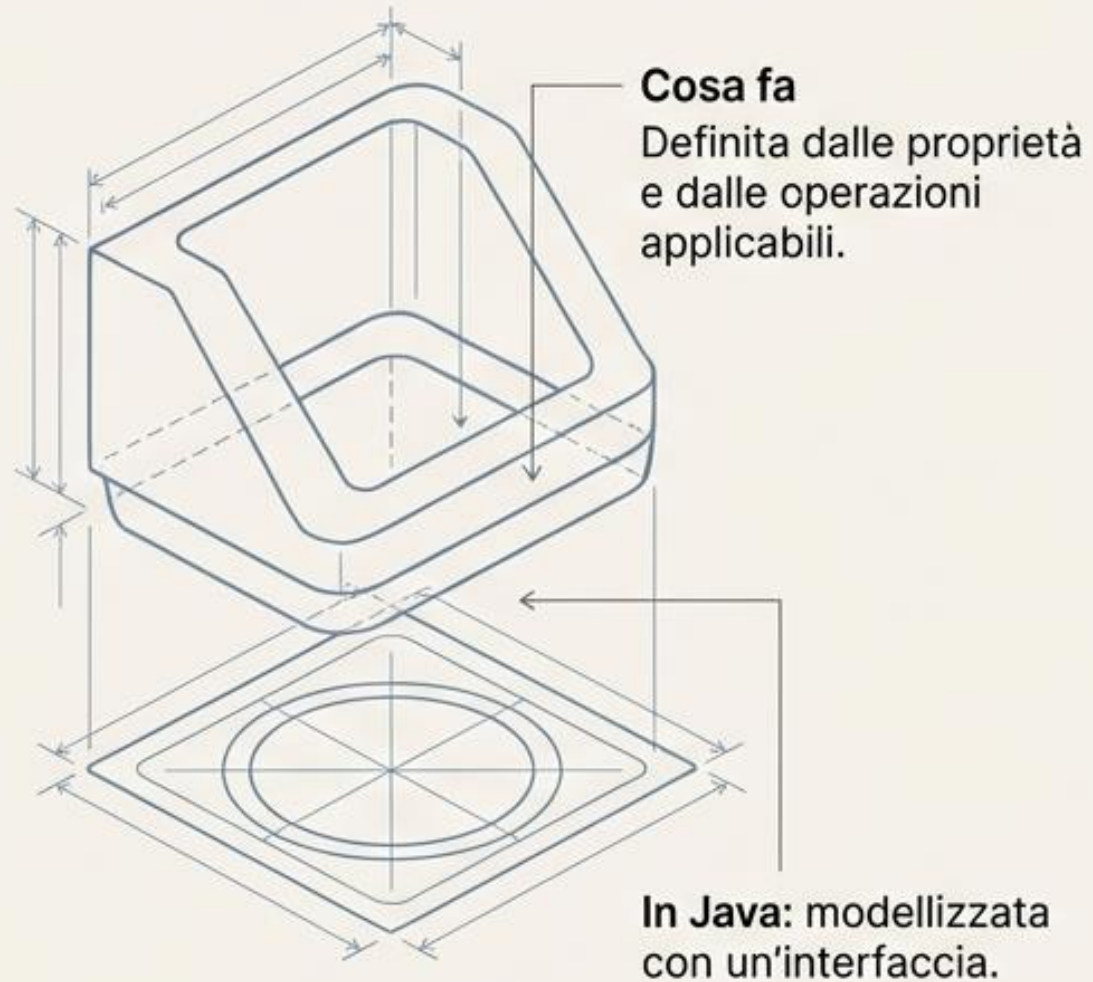
Strutture Dati Astratte

Dalla teoria dell'astrazione
all'implementazione concreta di
collezioni lineari e non lineari.

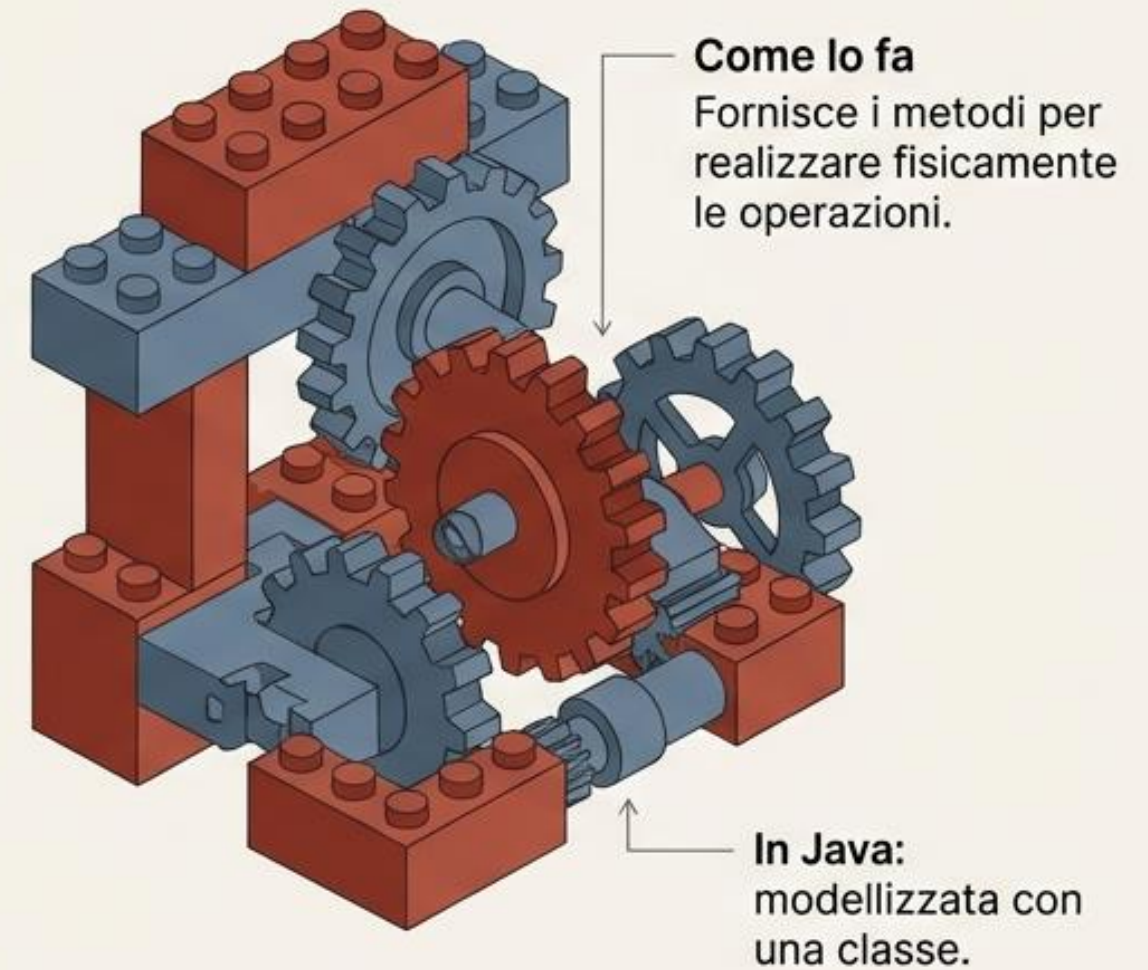


Il Paradigma dell'Astrazione

SDA (Struttura Dati Astratta)



Struttura Dati Concreta

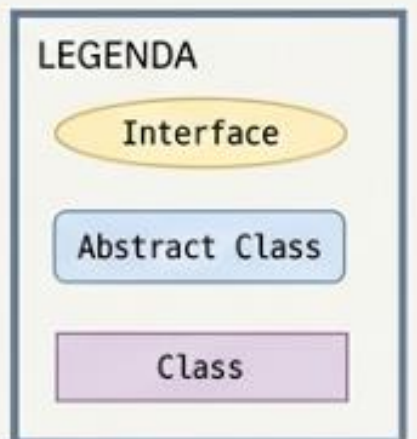
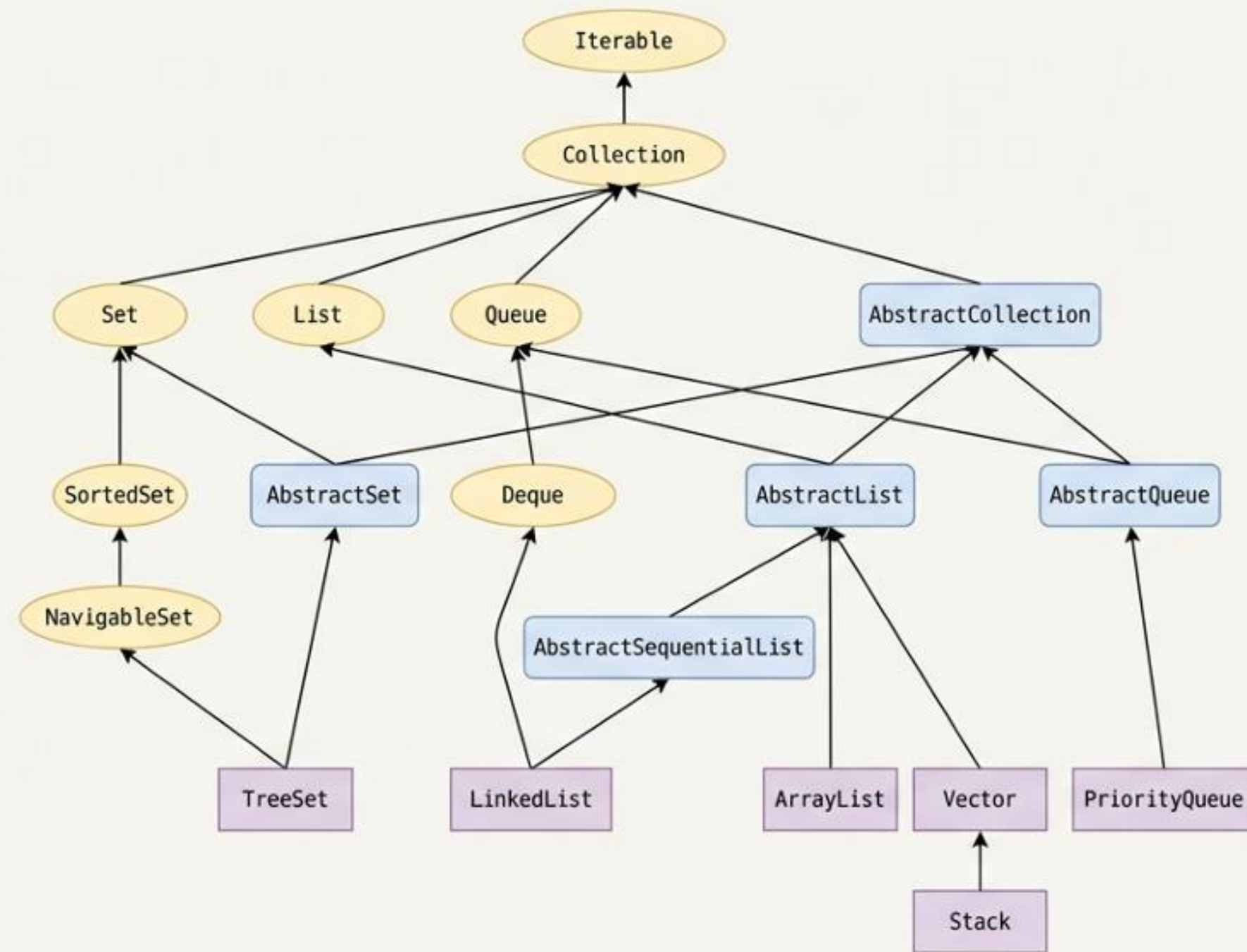


Esempio: `Vector` e `ArrayList` sono implementazioni concrete della SDA astratta `Array`.

- + Esistono numerose SDA, ad es. array, matrice, insieme, stringa, lista, pila, coda, albero, grafo, mappa, heap, etc.
- + Una SDA può essere implementata con diverse strutture dati concrete (ad es. Vector e ArrayList sono implementazioni di array); viceversa, una struttura dati concreta può implementare diverse SDA (ad es. ArrayList può essere usato per implementare array, stringhe, matrici, etc.)
- + Diverse implementazioni possono avere diversa efficienza

Il Quadro Completo del Framework

La gerarchia estesa delle Collezioni: come interfacce, classi astratte e implementazioni si fondono in un unico ecosistema.



Interfaccia Collection

- L'interfaccia specifica:
 - Operazioni di base
 - Operazioni più
 - specifiche

```
public interface Collection <E> {  
    // Basic Operations  
    int size();  
    boolean isEmpty();  
    boolean contains(Object element);  
    boolean add(<E> element);    // Optional  
    boolean remove(Object element); // Optional  
    Iterator <E> iterator();  
    boolean equals(Object o);  
    int hashCode();  
  
    // Bulk Operations  
    boolean containsAll(Collection <?> c);  
    boolean addAll(Collection<? extends E> c); // Opt.  
    boolean removeAll(Collection <?> c); // Optional  
    boolean retainAll(Collection <?> c); // Optional  
    void clear(); // Optional  
  
    // Array Operations  
    Object[] toArray();  
    <T> T[] toArray(T[] a);  
}
```

Interfaccia Set

Non permette di avere duplicati

- `s1.containsAll(s2)` $\Rightarrow S_1 \subseteq S_2$
- `s1.addAll(s2)` $\Rightarrow S_1 \cup S_2$
- `s1.retainAll(s2)` $\Rightarrow S_1 \cap S_2$

```
public interface Set <E> extends Collection <E>{
    // Basic Operations
    int size();
    boolean isEmpty();
    boolean contains(Object element);
    boolean add(<E> element);    // Optional
    boolean remove(Object element); // Optional
    Iterator <E> iterator();
    boolean equals(Object o);
    int hashCode();

    // Bulk Operations
    boolean containsAll(Collection <?> c);
    boolean addAll(Collection<? extends E> c); // Opt.
    boolean removeAll(Collection <?> c); // Optional
    boolean retainAll(Collection <?> c); // Optional
    void clear(); // Optional

    // Array Operations
    Object[] toArray();
    <T> T[] toArray(T[] a);
}
```


Interfaccia Iterator

- Un iteratore ha le seguenti funzionalità:
- `next()` che restituisce
 - l'elemento corrente della collezione, e contemporaneamente sposta il cursore all'elemento successivo;
 - `hasNext()` che verifica se il cursore ha ancora un successore se si è raggiunto la fine della collezione;
- `remove()` che elimina
 - l'elemento restituito dall'ultima
 - invocazione di `next()`;
 - `remove()` è opzionale perchè in certi casi non si vogliono mettere a disposizione
 - dell'utente metodi che permettano modifiche arbitrarie alla collezione.

```
public interface Iterator <E> {  
    boolean hasNext();  
    E next();  
    void remove();    // Optional  
}
```

Interfaccia List

- List può permettere di avere elementi duplicati (come Collection)
- List, oltre alle operazioni ereditate dal Collection include operazioni per:
- accesso in base alla
 - posizione
 - restituzione della posizione di un oggetto
 - restituzione di sottosequenze
- scansione bidirezionale del lista (mediante ListIterator)

```
public interface List <E> extends Collection <E>{
    /*...
    Metodi ereditati da Collection
    ...*/

    // Positional Access
    E get(int index);
    E set(int index, E element);          // Optional
    void add(int index, E element);      // Optional
    E remove(int index); // Optional
    boolean addAll(int ind, Collection <? extends E> c); //Opt

    // Search
    int indexOf(Object o);
    int lastIndexOf(Object o);

    // Iteration
    ListIterator <E> listIterator();
    ListIterator <E> listIterator(int index);

    // Range-view
    List <E> subList(int from, int to);
}
```


Interfaccia ListIterator

- Include le funzionalità di Iterator;
- Supporta la scansione inversa della lista (hasPrevious() e previous, analoghi a hasNext() e next());
- Restituisce la posizione dell'iteratore nella lista (nextIndex()) , (previousIndex());
- Permette la sostituzione dell'elemento corrente nella lista (set());
- Permette l'inserimento di un elemento nella lista (add()).

```
public interface ListIterator <E>
    extends Iterator <E>{
    /* Metodi ereditati da Collection:
    boolean hasNext();
    E next();
    void remove();
    */

    boolean hasPrevious();
    E previous();

    int nextIndex();
    int previousIndex();

    void set(E element);        // Optional
    void add(E element);        // Optional
}
```

Esempi di implementazione delle interfacce Set e List

- Collection non ha nessuna implementazione predefinita *diretta*.
- Tutte le sue implementazioni realizzano interfacce da essa derivate.
- Set è implementata dalla classe HashSet (HashSet <E> implements Set <E>, basata sull'uso di una **tabella hash**.
- List è implementata dalle classi:
 - ArrayList(ArrayList <E> implements List <E>), basata su un **array dinamico**. Costo della ricerca pari a $O(1)$, inserimento e cancellazione $O(n)$;
 - LinkedList(LinkedList <E> implements List <E>), basata su una **lista doppia** (con riferimento al successore ed al predecessore). Costo della ricerca, inserimento e cancellazione sono pari ad $O(n)$.
 - Inserimento/cancellazione in testa, coda, e durante la scansione dell'iteratore hanno costo $O(1)$.

Implementazione

- Useremo le classi tratte dal libro di testo e definite in <https://algs4.cs.princeton.edu/home/>
- Installare la libreria
 - algs4.jar

L'Ecosistema Lineare: Regole di Accesso

```
public class Bag<Item> implements Iterable<Item> {
```

Bag

```
private Item;  
private Item Item;  
public Item String();  
public void string();  
public Item();  
public QueueItem();
```



Collezione non ordinata.

Focus: Inserimento e
iterazione libera.

```
public class Stack<Item> implements Iterable<Item>
```

```
public interface <Item> {  
insertan ();  
Itemk> numerosstring();  
return Items();  
}
```

```
public class Queue<Item> implements Iterable<Item> {
```

Pila (Stack)

```
private  
private  
public Item String();  
public void string();  
public Item();  
public QueueItem();
```



Regola: LIFO
(Last In, First Out)

L'ultimo elemento inserito
è il primo ad uscire.

```
public class Stack<Item> implements Iterable<Item>
```

```
public interface  
insertan ();  
Itemk> numeros  
return Items();  
}
```

```
public class Queue<Item> implements Iterable<Item> {
```

Coda (Queue)

```
private  
private  
public Item String();  
public void string();  
public Item();  
public QueueItem();
```



Regola: FIFO
(First In, First Out)

Il primo elemento inserito
è il primo ad uscire.

```
public class Queue<Item> implements Iterable<Item>
```

```
public interface <Item> {  
insertan ();  
Itemk> numerosstring();  
return Items();  
}
```

Bag

```
public class Bag<Item> implements Iterable<Item>
```

Bag()	<i>create an empty bag</i>
void add(Item item)	<i>add an item</i>
boolean isEmpty()	<i>is the bag empty?</i>
int size()	<i>number of items in the bag</i>

FIFO queue

```
public class Queue<Item> implements Iterable<Item>
```

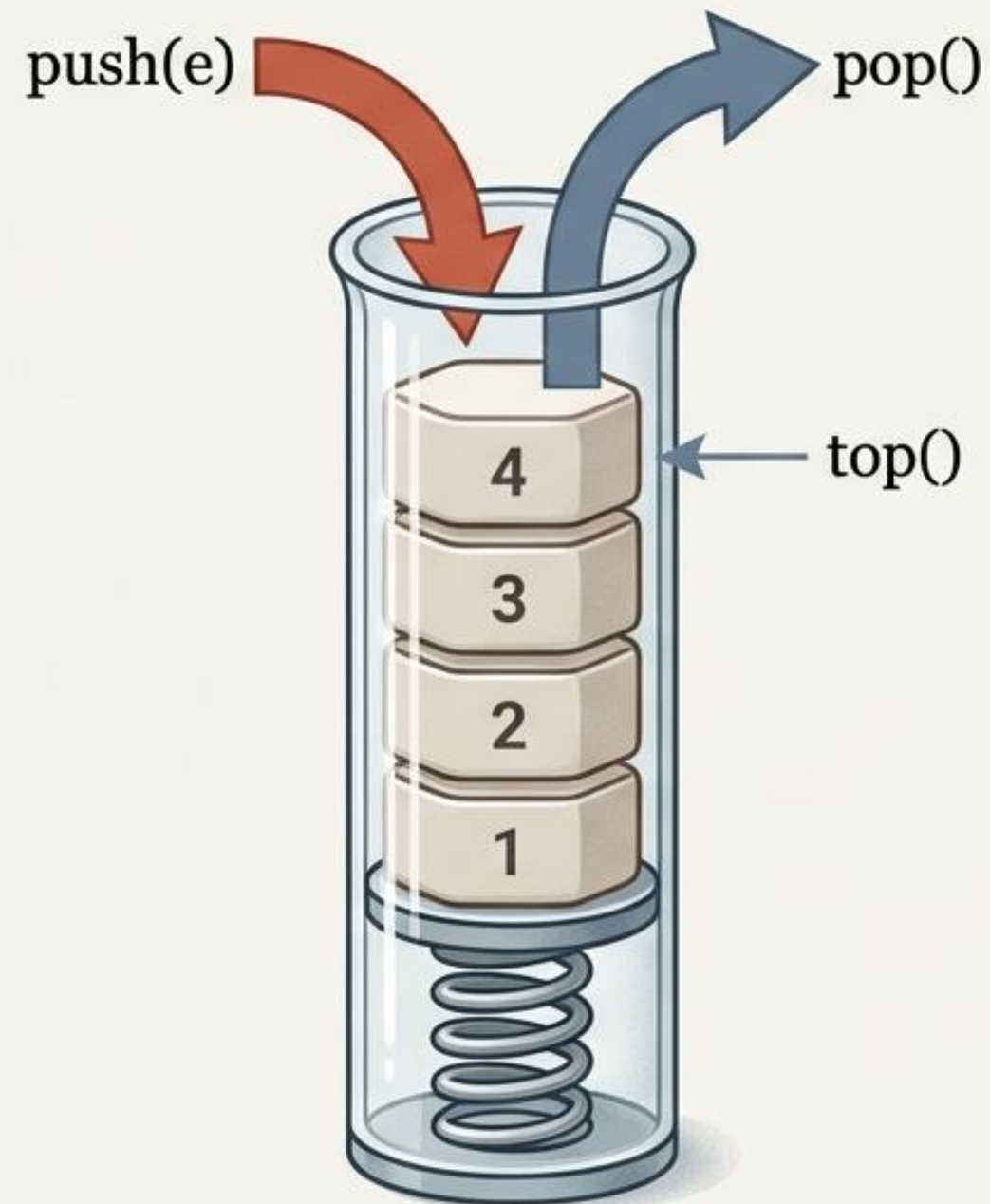
Queue()	<i>create an empty queue</i>
void enqueue(Item item)	<i>add an item</i>
Item dequeue()	<i>remove the least recently added item</i>
boolean isEmpty()	<i>is the queue empty?</i>
int size()	<i>number of items in the queue</i>

Pushdown (LIFO) stack

```
public class Stack<Item> implements Iterable<Item>
```

Stack()	<i>create an empty stack</i>
void push(Item item)	<i>add an item</i>
Item pop()	<i>remove the most recently added item</i>
boolean isEmpty()	<i>is the stack empty?</i>
int size()	<i>number of items in the stack</i>

Meccanica della Pila (Stack)



Operazioni Core

`push(e)`: Inserisce un elemento in testa.

`pop()`: Rimuove l'elemento dalla testa.

`top()`: Legge l'elemento in testa senza rimuoverlo.

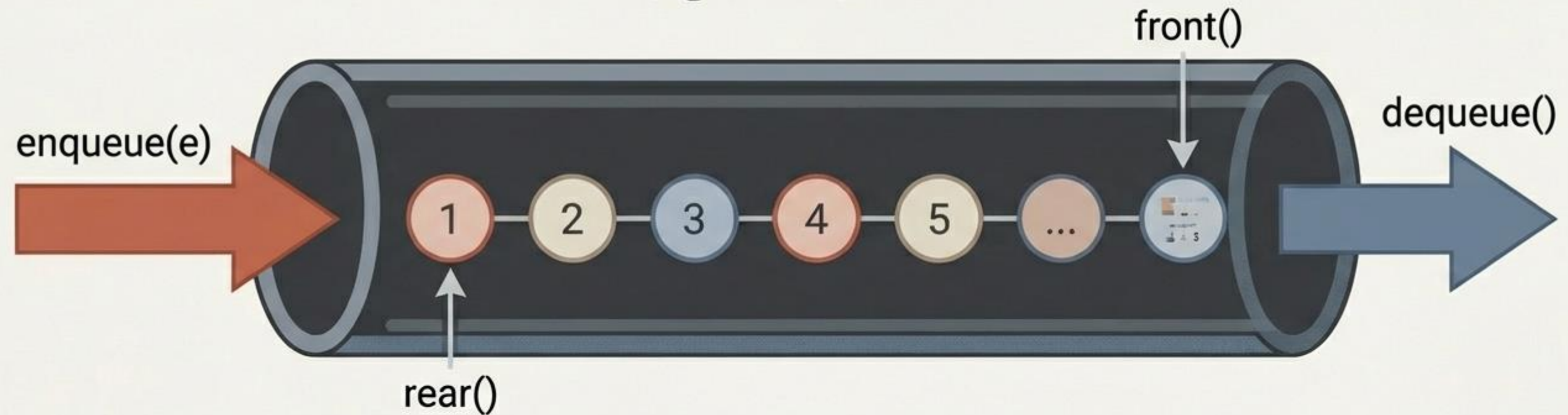
`size()` / `empty()`: Metriche di stato.

Esempi Reali

- Record di attivazione (chiamate di funzioni).
- Cronologia del browser web (pulsante Indietro).
- Meccanismo di Undo negli editor di testo.

Operation	Output	Stack Contents
push(5)		push (5)
push(3)		push (3)
pop()	None	push (7)
push(7)		push (3)
pop()	None	push (5)
top()		4, (4)
pop()		4, (3)
pop()		3, (2)
top()		4, (1)
empty()	None	0
push(9)		4, (9)
push(7)		4, (7)
push(3)		3, (3)
push(5)		1, (1)
size()	None	6
pop()		4, (5)
push(8)		4, (8)
pop()		3, (3)
top()		1

Meccanica della Coda (Queue)



Glossary

`enqueue(e)`: Inserisce in coda (`rear`).

`dequeue()`: Rimuove dalla testa (`front`).

`front()`: Legge l'elemento in testa.

`rear()`: Legge l'elemento in coda.

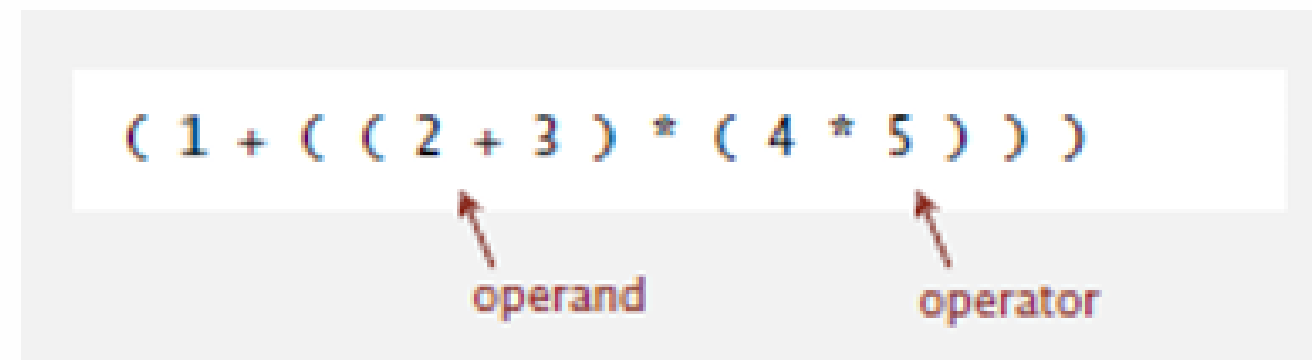


Costo Costante.

Poiché è possibile accedere a entrambe le estremità della struttura, ogni operazione viene eseguita in tempo costante.

QUALCHE APPLICAZIONE

- Scrivere un algoritmo che valuti un'espressione aritmetica in forma infissa

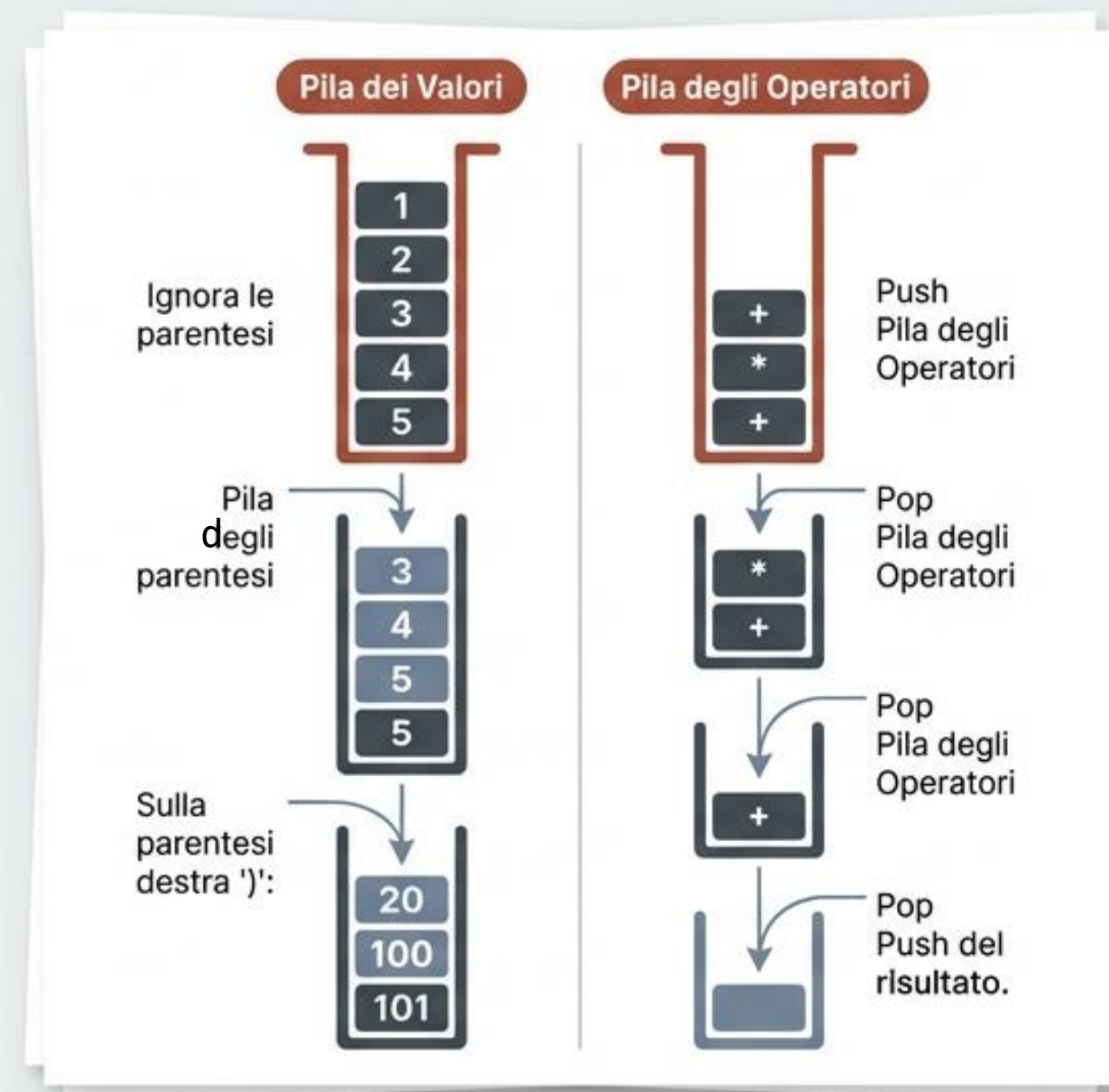


Algoritmo in Azione: Valutazione con 2 Pile

(1 + ((2 + 3) * (4 * 5)))

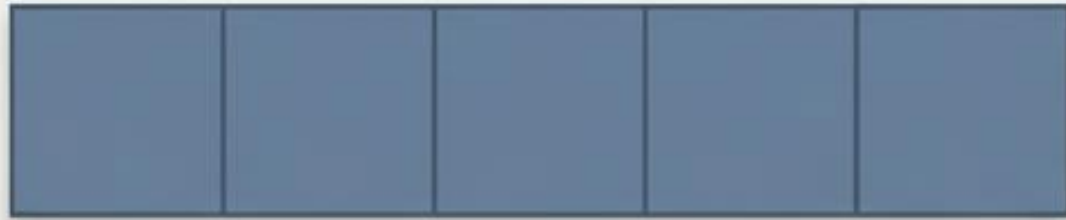
operand operator

1. Ignora le parentesi sinistre '('.
2. Valori inseriti nella Pila dei Valori.
3. Operatori inseriti nella Pila degli Operatori.
4. Sulla parentesi destra ')': Pop operatore, Pop valori corrispondenti, calcola, e Push del risultato.



Implementazione Concreta: Array vs. Lista Concatenata

Array



Pro: **Accesso rapido** agli elementi (indici contigui in memoria).

Contro: Richiede la gestione manuale del ridimensionamento (**resizing**). La rimozione in testa è lenta, poiché richiede di riscalarare tutti gli elementi successivi.

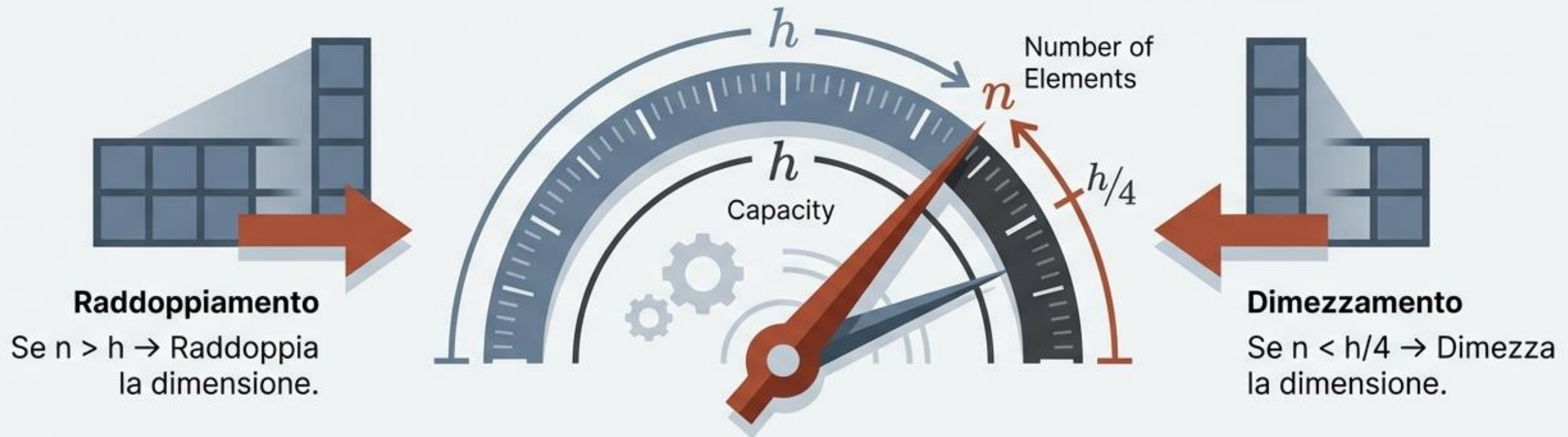
Lista Concatenata



Pro: **Inserimento in coda molto veloce** e dinamico; non richiede mai procedure di resizing.

Contro: Accesso sequenziale più lento a causa della **memoria frammentata**.

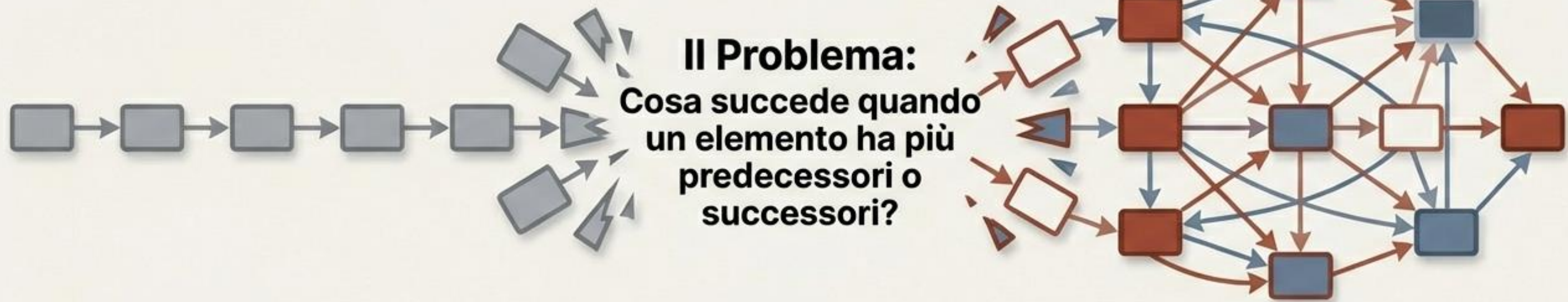
La Matematica del Resizing: Raddoppiamento e Dimezzamento



$$n \leq h \leq 4n$$

Garantisce che la taglia dell'array sia sempre vicina al numero di elementi da contenere, ottimizzando il costo ammortizzato a $O(1)$.

Oltre la Linearità: Dalle Sequenze alle Reti



Biologia
(Strutture molecolari)

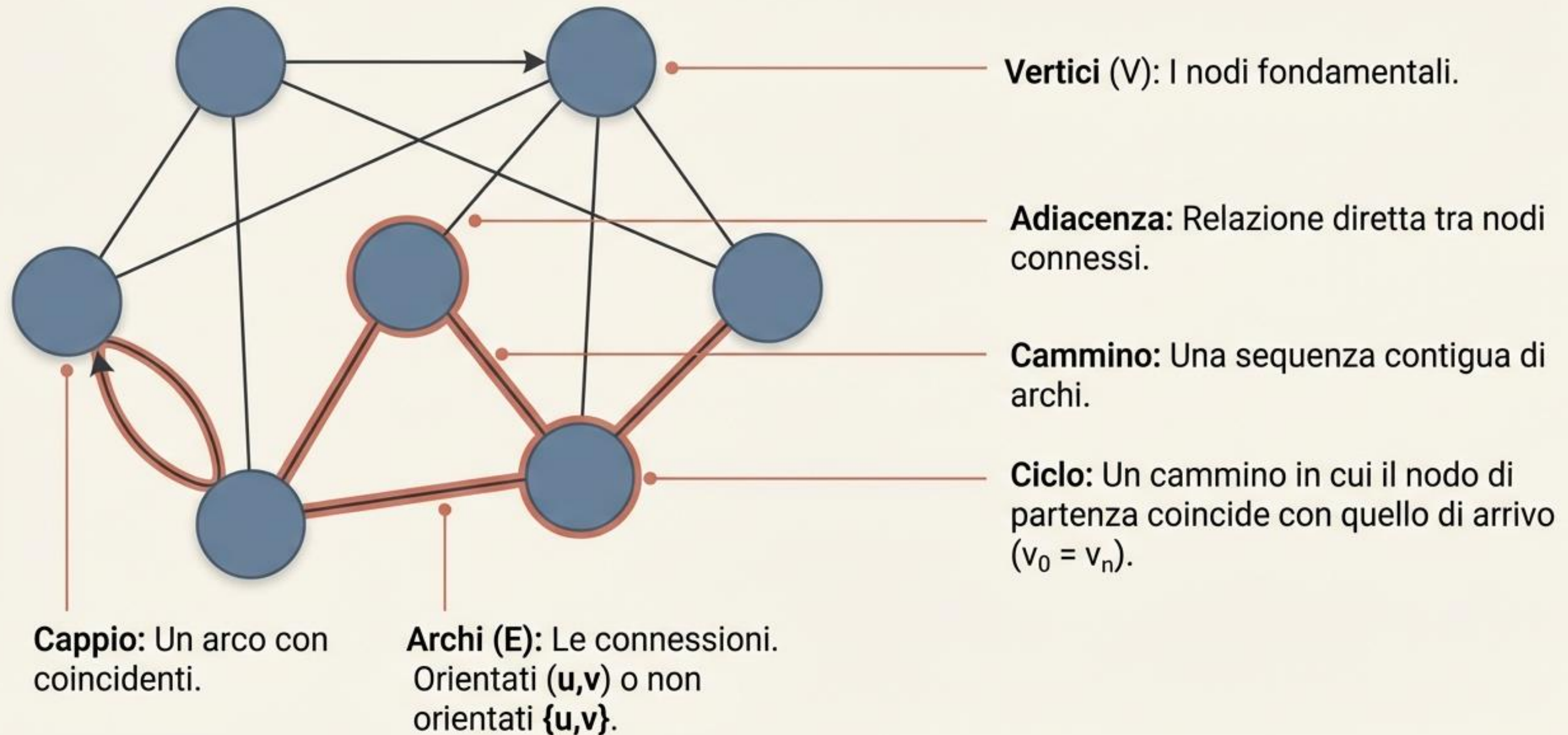


Geografia
(Sistemi fluviali, reti stradali)

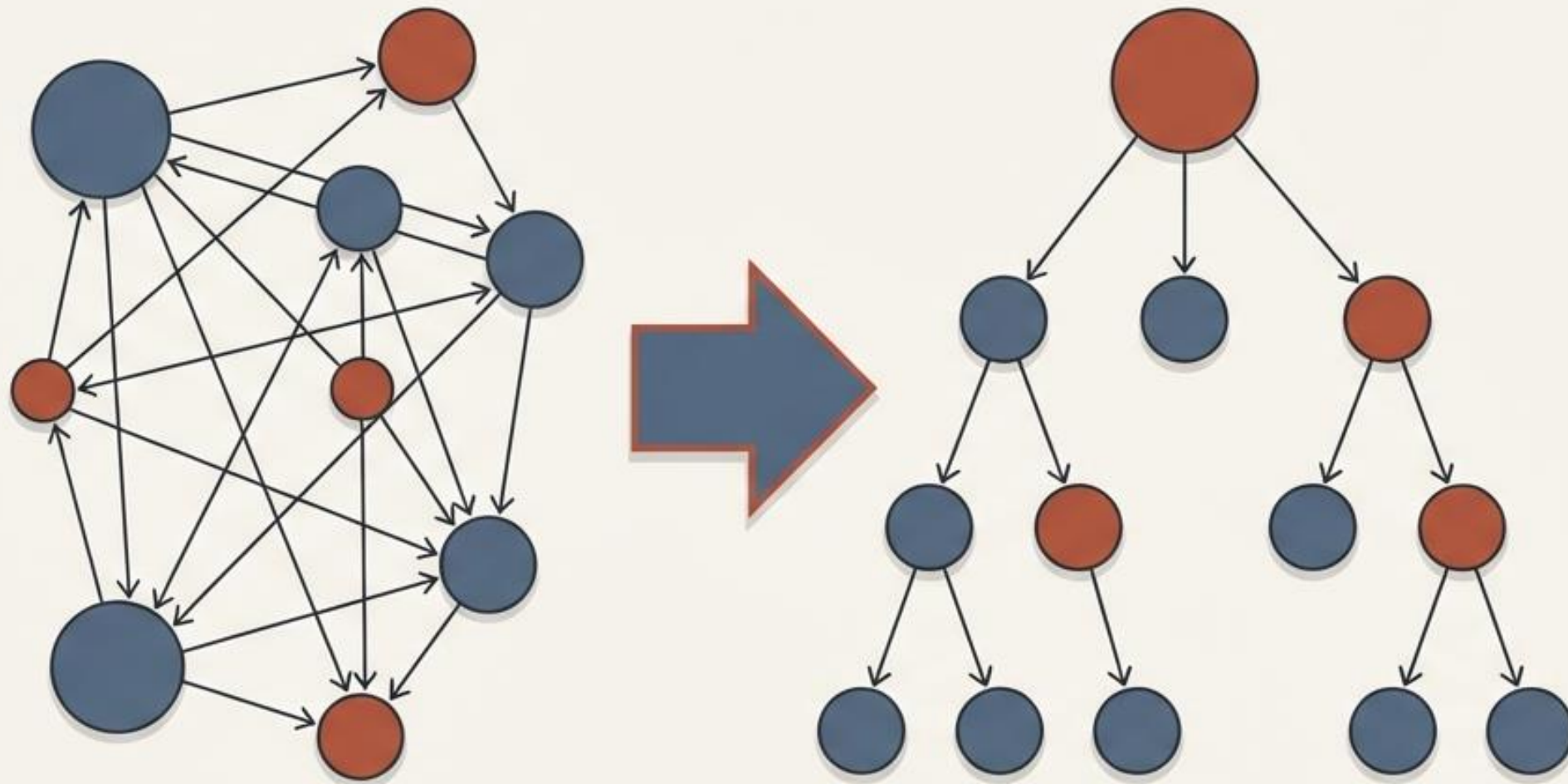


Storia
(Alberi genealogici, filologia)

Fondamenti di Teoria dei Grafi



L'Albero: Un Grafo Gerarchico

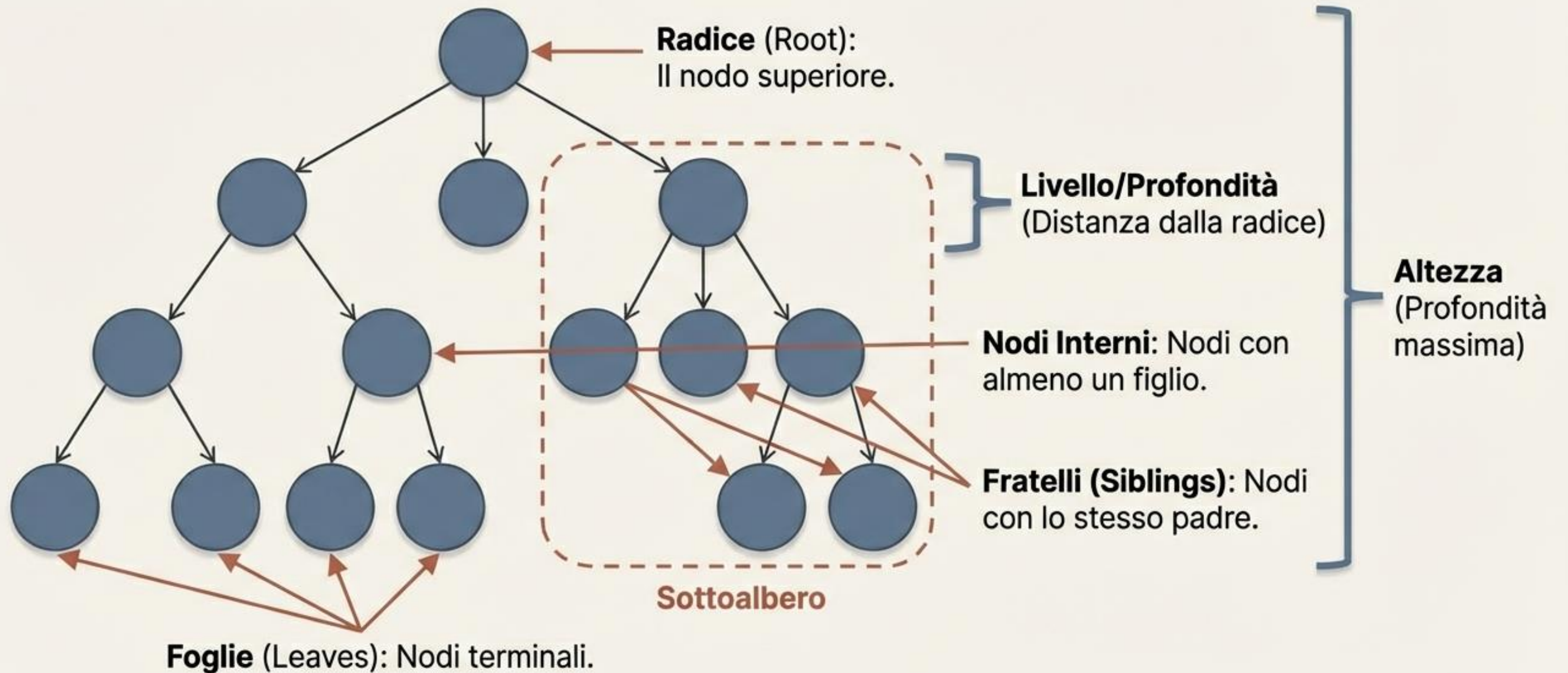


Le 3 Regole d'Oro

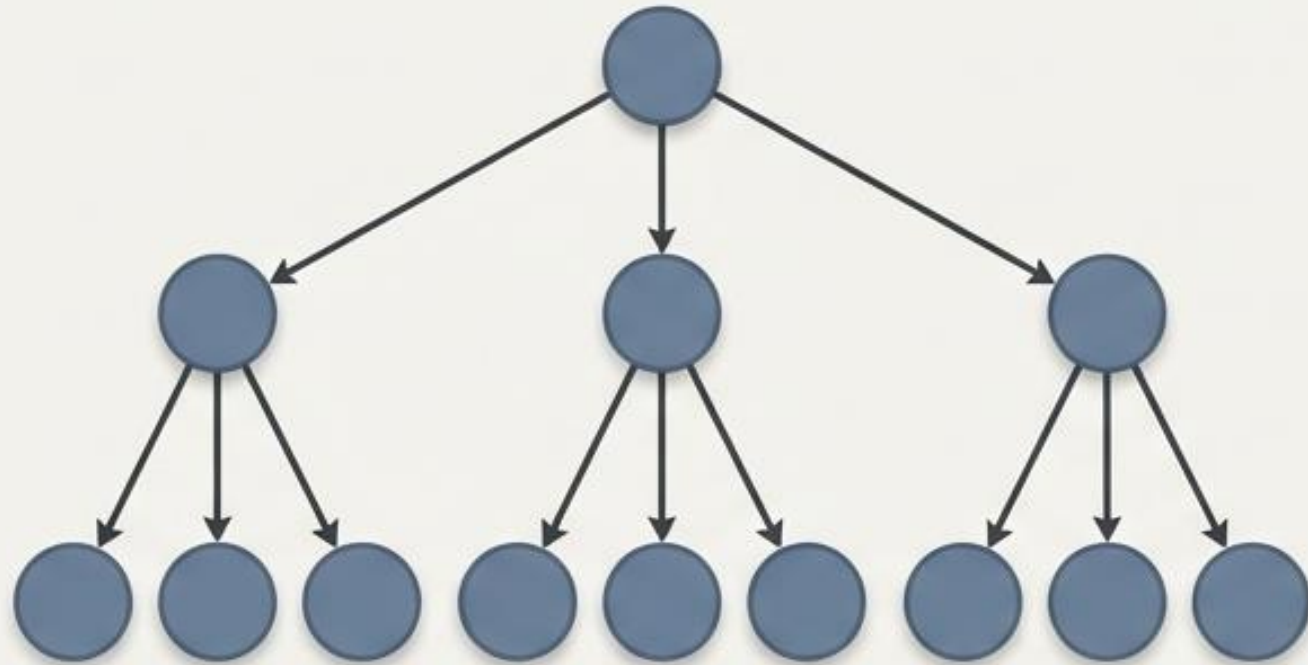
- 1. Formula rigorosa:**
 $|E| = |V| - 1$
- 2. Aggiungere un arco**
crea inevitabilmente
un ciclo.
- 3. Togliere un arco**
rompe la connessione.

Queste proprietà garantiscono che esista sempre un unico, singolo cammino tra due nodi qualsiasi nell'albero.

Anatomia e Metriche di un Albero

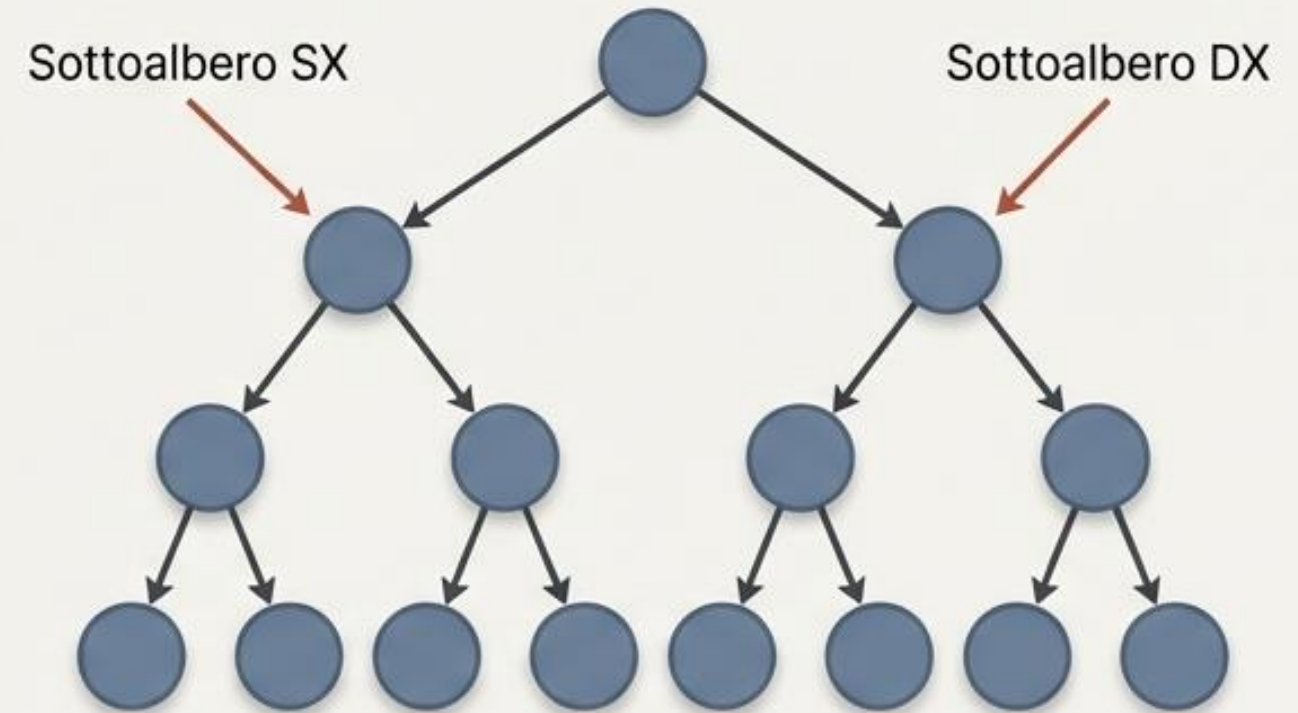


La Famiglia degli Alberi: K-ari e Binari Completi



Albero K-ario

Tutte le foglie hanno la stessa profondità e tutti i nodi interni hanno grado k .



Albero Binario

Albero posizionale: ogni nodo ha al più due figli.

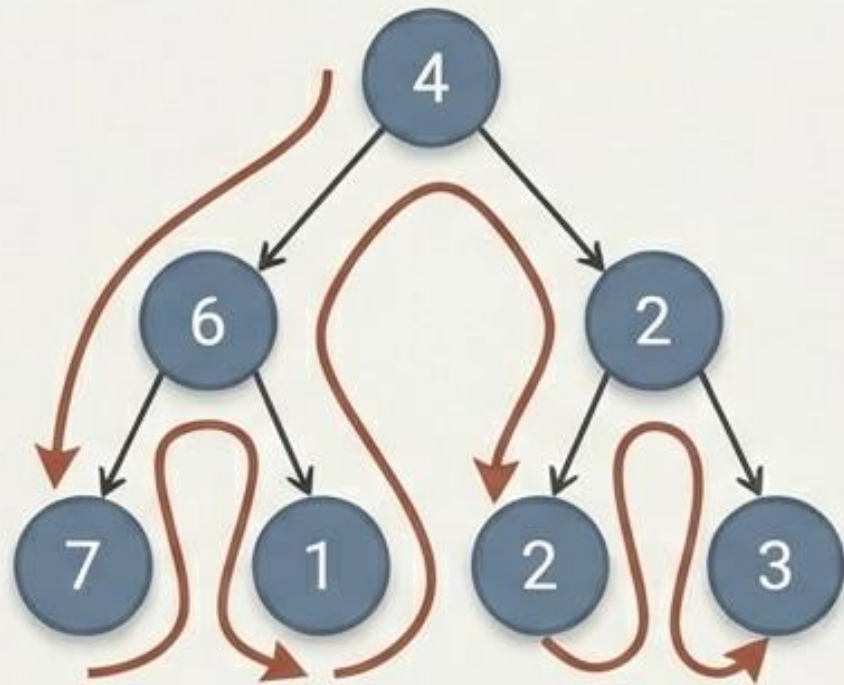
Formula Box

Matematica dell'Albero Binario (altezza h):

- Foglie totali: 2^h
- Nodi interni: $2^h - 1$
- Nodi totali: $2^{(h+1)} - 1$

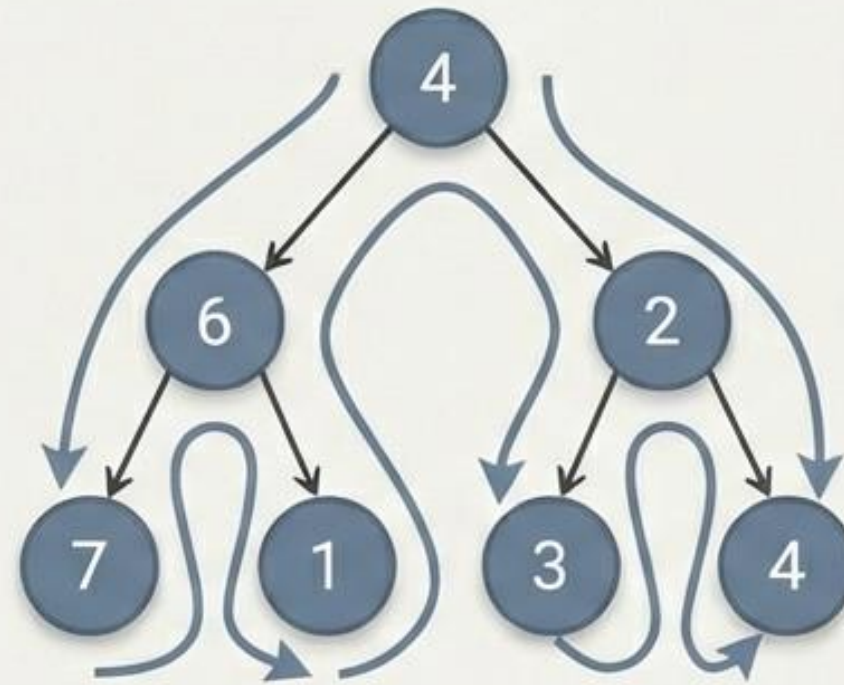
Performance: Altezza $O(\log n)$ rispetto a $O(n)$ lineare.

Navigare l'Albero: Le Visite



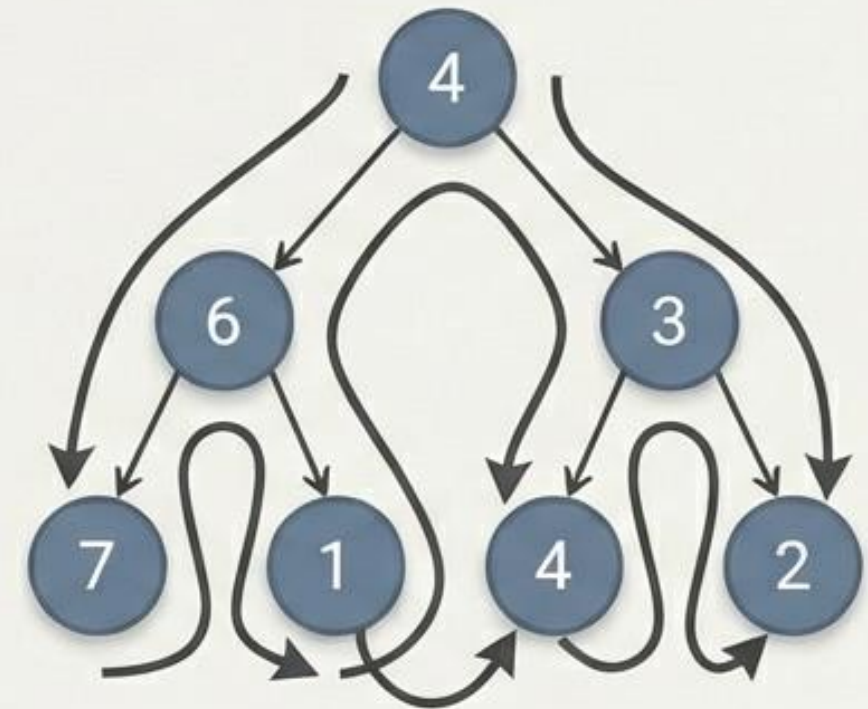
Visita in Pre-ordine

Sequenza: Radice, SX, DX



Visita in Post-ordine

Sequenza: SX, DX, Radice



Visita in Ordine Simmetrico

Sequenza: SX, Radice, DX

Esempio Output: Visita in ordine simmetrico: 7 6 1 4 2 3

Sfide Algoritmiche: Mettiti alla Prova

1. Strutture Lineari

Bilanciamento Parentesi: Usa una Pila per validare stringhe complesse come `[]{}X[()()()]()`.

2. Liste Concatenate

Riordino: Scrivi una funzione che riordini una lista separando gli indici pari da quelli dispari.

3. Alberi (Ricostruzione)

Costruisci l'albero esatto date esclusivamente le sue sequenze di visita in pre-ordine e in ordine simmetrico.

4. Metriche e Alberi

Trasformazione: Calcola l'altezza di un albero binario e trasformalo nel suo esatto speculare.

La padronanza delle strutture dati si ottiene trasformando l'astrazione in implementazione.